Painting with Darts: Language-Model Orchestration of a Fine-Tuned $\pi_{0.7}$ Policy on a Low-Cost Arm

Vineeth Sendilraj Vardhan Agnihotri Yuvraj Lakhotia

Act-athon 2026, Actor Labs × Physical Intelligence (3rd place)

***Abstract*—Vision-language-action (VLA) models can learn a new manipulation skill from a small number of demonstrations, but a policy that follows one instruction at a time does not, by itself, plan toward a multi-step goal or notice when a step has failed. We study a simple hierarchy for a long-horizon task whose outcome is visible on a target: drawing a picture on a 5×5 grid by firing foam darts from a motorized blaster held by a low-cost SO-101 arm. A language model (Claude) acts as the orchestrator. It turns a request such as "draw the PI logo" into a sequence of target cells, inspects the board after every shot, and re-issues shots that miss. A fine-tuned $\pi_{0.7}$ policy executes each command, mapping "shoot ⟨cell⟩" and two camera views to joint trajectories and a trigger signal at 30 Hz. The policy was trained on 116 teleoperated demonstrations (16 minutes of data) collected in a single session. In an offline probe, every fine-tuned checkpoint aimed in the commanded row and column order, and the full system completed a live drawing demonstration. The dataset, code and hardware design are available at `https://github.com/VineethSendilraj/pi07-dart-painting-so101`.**

I. Introduction

Recent VLA models such as π_0 [4] and $\pi_{0.7}$ [6] can be adapted to a new robot and task with tens of minutes of teleoperated data. Many useful tasks, however, are not a single command but a sequence whose next step depends on how the previous ones went. A common way to handle this is a hierarchy in which a high-level model chooses the next instruction and a low-level policy carries it out [1, 2].

We built such a system over a weekend hackathon around a deliberately unforgiving task. A dart blaster held by an SO-101 arm must hit specific cells of a grid in order to draw a shape. Each shot either lands in the right cell or it does not, the result is visible to a camera, and darts sometimes bounce off. This makes the outcome easy to check visually but hard to guarantee from the policy alone.

Our contributions are modest and practical: (i) a working two-level system in which Claude plans and verifies shots while a fine-tuned $\pi_{0.7}$ policy aims and fires; (ii) a hardware recipe that turns a commercial motorized blaster into a single digital output; and (iii) an open dataset of 116 demonstrations in LeRobot format [8], released with the code and wiring.

II. Related Work

Language models over robot skills. SayCan [1] grounds a language model's proposals in the value functions of pretrained skills, and Hi Robot [2] uses a high-level VLM to issue language commands to a π_0 policy and to incorporate user feedback. We follow the same split but use a general-purpose model without fine-tuning, and close the loop with images of the task outcome rather than skill values.

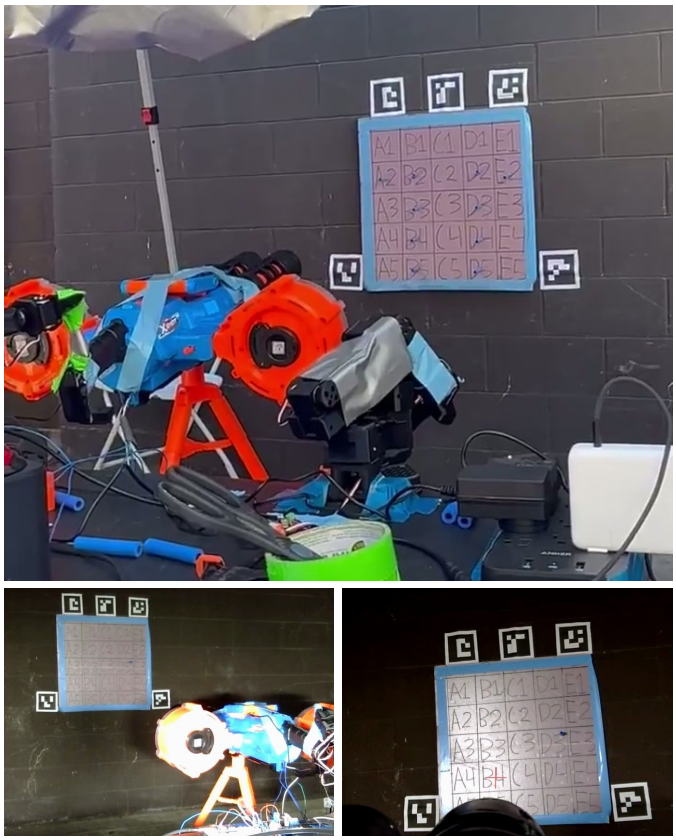


Fig. 1. Top: the rig. The SO-101 arm moves the handle of a motorized dart blaster mounted on a tripod, facing a 5×5 foam board. Bottom: the policy's two camera views during a `shoot B4` demonstration. The rear camera (left) sees the arm, blaster and board; the wrist camera (right) is mounted on the blaster, and the red cross marks the barrel axis.

VLA policies. RT-2 [3] showed that vision-language pretraining transfers to robot control. π_0 [4] adds a flow-matching action expert, $\pi_{0.5}$ [5] co-trains on heterogeneous data for open-world generalization, and $\pi_{0.7}$ [6] conditions on richer prompts. We fine-tune $\pi_{0.7}$ on a single narrow skill.

Low-cost teleoperation. Leader-follower teleoperation with inexpensive arms was popularized by ALOHA [7]; the SO-101 and the LeRobot library [8] make the same setup available for a few hundred dollars.

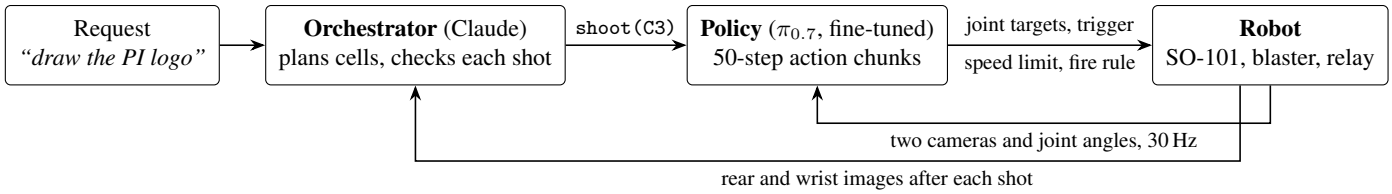


Fig. 2. System overview. The orchestrator operates at the level of shots (one decision every few seconds); the policy operates at the level of joint commands (30 Hz).

III. System

A. Hardware

The blaster is an off-the-shelf motorized dart blaster mounted on its tripod. We removed the SO-101 follower's gripper and connected the arm to the blaster handle with a loop of wire through holes drilled in both; the slight compliance turned out to help the arm move the handle smoothly. The blaster's fire microswitch is replaced by the changeover contacts of a relay driven by an ESP32, so firing becomes one digital output. A rocker switch in the battery line serves as master power: with it off, the relay still clicks and the trigger label is still recorded, which allowed dry demonstrations with identical motion. Darts carry thumbtack tips so that hits stay in the foam. Two USB cameras record at 640×480 and 30 Hz (Fig. 1).

B. Policy

At each request the policy receives both camera images, the five follower joint angles and the prompt `shoot ⟨cell⟩`. It returns a chunk of 50 future steps, each containing five absolute joint targets and a trigger value. We execute 25 steps before querying again, and limit every joint to 3° per step. A shot is fired only if the arm has stopped moving and the predicted trigger exceeds 0.5 for three consecutive executed steps, or if the remainder of the chunk holds the current aim and contains such a run.

C. Orchestrator

Claude receives the user's request and two photographs: the rear view, which shows which cells already hold darts, and the wrist view, which shows the cell under the cross. It responds with one tool call per turn, either `shoot(cell)` or `finish`. After each shot it receives new photographs, so a dart that missed its cell or fell out of the board is detected and the cell is shot again (Fig. 2). The orchestrator is told which cells have no training data and avoids them.

IV. Dataset

An operator aimed the follower using the leader arm and fired with a physical button wired to the ESP32. A small web application proposed the next cell, board clutter and start pose, drew the cross on the wrist view, and replayed every take frame by frame so that it could be kept or discarded. Table I summarizes the 116 kept demonstrations. The trigger label is 1 for the 10 steps after the button press and 0 otherwise; 23 shots were fired with live darts and 93 were dry.

TABLE I
Dataset summary

Demonstrations	116 (29,131 frames, 16.2 min, 30 Hz)
Episode length	142–505 steps
Cells covered	23 of 25 (A5, E5 absent)
Observations	rear and wrist RGB, 5 follower joints
Actions	5 leader joints, trigger
Variation	live/dry, 3 clutter levels, 17 start poses
Format, license	LeRobot v3.0, CC BY 4.0

V. Training and Evaluation

We fine-tuned $\pi_{0.7}$ through Physical Intelligence's hosted fine-tuning service, running 13 jobs that varied the number of steps, batch size, data mixture and base checkpoint. Table II lists the directly comparable runs, which used the standard base checkpoint on the full dataset; all 13 runs appear in Table III in the appendix. Validation error was lowest after roughly 30–35 passes over the data and increased with longer training; a larger batch did not help, nor did oversampling the cells of the PI logo. An alternative base checkpoint offered by the service (“cosmic”) peaked earlier, at about 18 passes, and reached similar error. Keeping every demonstration was better than training only on the 101 takes recorded with the final crosshair.

TABLE II
Validation error, $\pi_{0.7}$ on all 116 demonstrations

Run	Steps	Passes	Val. error
Baseline	3,000	26.5	0.298
Longer schedule	3,800	33.5	**0.287**
Much longer	5,500	48.5	0.324
Batch 512	2,750	48.5	0.334
Logo cells oversampled	3,800	33.5	0.333

To check that the policy uses the instruction, we fed every checkpoint the same rig images and joint state while changing only the cell in the prompt, and compared the final aim of each predicted chunk. All 24 checkpoints produced aims ordered by column and row, so the policy follows the language command rather than replaying a fixed motion. We used the best run in Table II for the live demonstration, in which the system drew its target picture.

VI. Limitations

We did not measure a hit rate on the real system; our evidence is offline validation error, the prompt probe and the live demonstration. Every demonstration continues for 2 s after the shot with

TABLE III
ALL FINE-TUNING RUNS

Run	Base	Change from baseline	Steps (passes)	Val. error (step)	Note
full-v1	$\pi_{0.7}$	none (baseline)	3,000 (26.5)	0.298 (2,999)	
full-v2	$\pi_{0.7}$	longer schedule	3,800 (33.5)	**0.287** (3,799)	used in the live demonstration
full-long-v1	$\pi_{0.7}$	much longer schedule	5,500 (48.5)	0.303 (4,000)	overfits after $\sim$35 passes
full-long-b512-v1	$\pi_{0.7}$	batch 512	2,750 (48.5)	0.334 (2,749)	
logo-v2	$\pi_{0.7}$	50% of each batch from the 11 logo cells	3,800 (33.5)	0.333 (3,799)	
redmark-v1	$\pi_{0.7}$	only the 101 takes with the final crosshair	4,600 ($\sim$47)	0.325 (4,000)†	
noidle-v1	$\pi_{0.7}$	still frames dropped from training	3,000 (26.5)	0.421 (2,999)‡	
full-cosmic-v1	cosmic	none	3,000 (26.5)	0.294 (2,000)	
full-cosmic-2k-v1	cosmic	shorter schedule	2,000 (17.6)	0.294 (1,999)	
logo-cosmic-v1	cosmic	50% of each batch from the 11 logo cells	2,000 (17.6)	0.281 (1,999)§	only clean offline trigger pulse
redmark-cosmic-v1	cosmic	only the 101 takes with the final crosshair	4,600 ($\sim$47)	0.294 (2,000)†	
noidle-cosmic-v1	cosmic	still frames dropped from training	2,000 (17.6)	0.375 (1,999)‡	
logo-noidle-cosmic-v1	cosmic	logo oversampling and still frames dropped	2,000 (17.6)	0.444 (1,999)‡	

†Validated on the 101-take subset. ‡Validated on moving frames only. §Lowest overall, but its validation split likely over-represents the logo cells. Two further runs were cancelled before completion, and a run on trimmed takes was planned but not trained. All 24 checkpoints probed with the prompt test aimed in column and row order.

the arm on target and the trigger at 0, so most on-target frames are labeled "do not fire." Offline, the policy rarely produced a clean trigger pulse, which is why the fire rule in Section III-B checks for a sustained trigger only once the arm is still. Coverage is uneven (two demonstrations each for row 1, A3 and E3), and the camera and board positions were fixed throughout, so we do not expect the policy to transfer to a different layout without new data.

ACKNOWLEDGMENTS

We thank Actor Labs and Physical Intelligence for organizing the Act-athon and providing API access, and the organizers and judges for their help during the event.

REFERENCES

[1] M. Ahn *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," arXiv:2204.01691, 2022.

[2] L. X. Shi *et al.*, "Hi Robot: Open-ended instruction following with hierarchical vision-language-action models," in *ICML*, 2025.

[3] A. Brohan *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," arXiv:2307.15818, 2023.

[4] K. Black *et al.*, "π_0: A vision-language-action flow model for general robot control," in *RSS*, 2025.

[5] Physical Intelligence *et al.*, "$\pi_{0.5}$: A vision-language-action model with open-world generalization," arXiv:2504.16054, 2025.

[6] Physical Intelligence *et al.*, "$\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities," arXiv:2604.15483, 2026.

[7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *RSS*, 2023.

[8] R. Cadene *et al.*, "LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch," `https://github.com/huggingface/lerobot`, 2024.

APPENDIX: ALL FINE-TUNING RUNS

Table III lists every fine-tuning run. Validation error is computed by the fine-tuning service on its own held-out split, at the checkpoint shown. Scores marked † or ‡ use a different validation set and should not be compared with the rest.